

AMLS SoSe 2026 — AI Image Detection

Marco Reczuch

David Birgel

Jerome Engelbrecht

July 9, 2026

1 Introduction and Data

Within this project, we have built a CPU dedicated pipeline capable of classifying given images as being *real* (0) or generated by AI models (1), five generators (SD 2.1, SDXL, SD 3, DALL-E 3, Midjourney) are combined into class 1. The data that has been labelled (29688 train, 1924 calibration, 1124 validation images, plus augmented counterparts) is made of JPEG images. The unlabelled `predict/` split score gets calculated at the inference step. The pipeline contains six required stages (`clean`, `prepare`, `train`, `predict`, `train_augmented` and `predict_augmented`) that all share one module (`amls_common`, `amls_model`). In that way, training and inference can use byte-identical preprocessing. The randomness used in the project is seeded and the CPU threads are capped at 8.

2 Task 1.1 — Dataset Exploration and Cleaning

Class Distribution. We balanced the training set across the six source classes (with about 4948 cases for each). This represents about a 1:5 real to AI ratio after merging (Fig. 1 left). We will keep this ratio and address it when we get to the modelling through class-weighted loss and calibrated threshold. We prefer this instead of re-sampling, because the evaluation splits have the same imbalance.

The important characteristic is aspect ratio. When we inspect image sizes, it reveals a close to perfect shortcut in train, real (COCO) images are nearly always non-square, e.g. 640×480 or 640×427 however nearly all AI images are square (320×320 or 270×270). Basically, amongst all the kept real images 4741 are non-square where only 146 are square, while all 24489 AI images are in fact square (Fig. 1 right). However, in the calibration, the validation and the predict splits, we can see they are uniformly 320×320 . We can infer that aspect ratio is an artifact linked only to training, making a model that relies only on that metric would label all square validation images as AI (possibly a 100% false-positive rate on real images). We remove this shortcut, by changing every training image resolution to a square, and this is the most important cleaning decision for this training case, we match the test distribution.

There are other tells, firstly, all splits are in fact JPEG/RGB, that means file format carries no important signal, since the dataset authors already appear to have normalized it. Secondly, the averaged FFT power spectrum (Fig. 2a) indicates that real images usually carry consistently higher frequency energy than AI images. This is a real forensic cue that happens to also be present on validation, so it transfers. Third, we can notice that AI images are slightly brighter and less saturated. And fourth, the augmented splits have smaller encoded sizes (Fig. 2b), indicating the augmentation pipeline applies stronger JPEG compression. As has been shown in Task 1.3, this mostly destroys the high-freq. cue and motivates augmentation based robustness.

Descriptive statistic (training)	Real (0)	AI (1–5)
images	4887	24489
typical resolution	640×480 non-square	320^2 and 270^2 square
mean encoded size	50 KB	26 KB
mean brightness (RGB)	0.43	0.47
mean saturation	0.29	0.28

Table 1: These class related descriptive statistics show us that resolution and encoded size are strongly correlated with the label in train, but are in fact neutralized by the square resizing approach. Colour differences are weak and Counts are after de-duplication.

Deterministic cleaning pipeline (`clean.py`). We decoded all training images and dropped undecodable streams (0 found), we also dropped exact duplicates via an MD5 of the encoded bytes while keeping the first occurrence (312 removed), and we recorded per-image metadata. The obtained result is a deterministic cleaned index of 29376 rows. Squaring to the model resolution, justified above, is applied in `prepare.py`. We intentionally avoid any non-deterministic cleaning, or cleaning that alters the content, such as denoising, that could erase the forensic cues the detector should use.

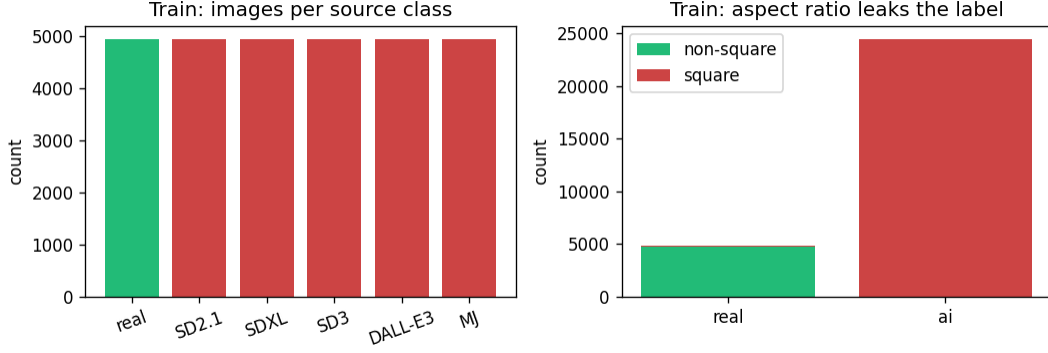
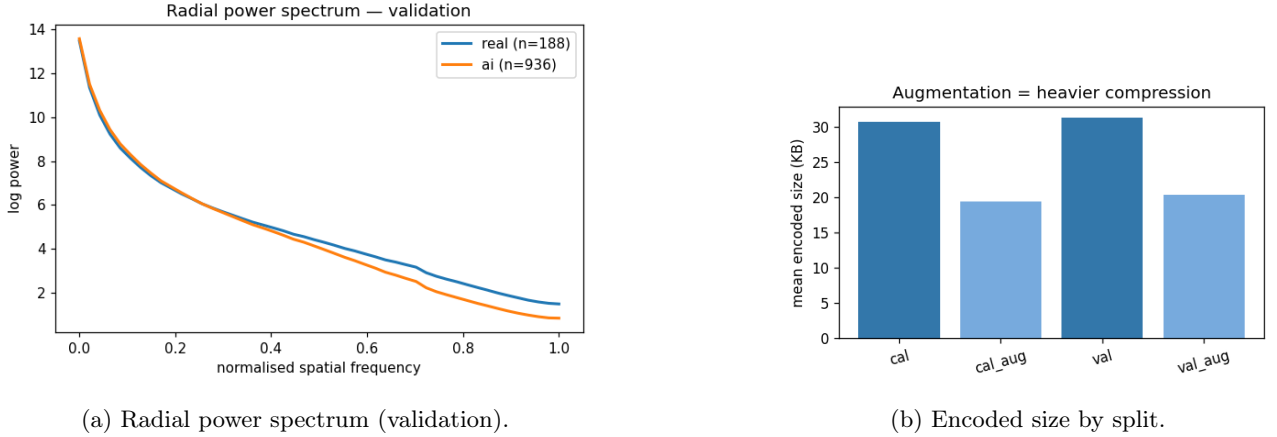


Figure 1: Left: balanced source-class counts in train. Right: in train, real images are non-square and AI images are square this is a shortcut absent from the 320² evaluation splits.



(a) Radial power spectrum (validation).

(b) Encoded size by split.

Figure 2: Forensic cues. Real images keep more high-frequency energy in the radial power spectrum on validation, augmentation compresses images more heavily, as shown by encoded size by split.

3 Task 1.2 — Modelling and Tuning under Time Constraints (35 pts)

Protocol. We maximise recall on AI images subject to a false-positive rate (FPR) on real images of at most 20%. The operating threshold is *calibrated automatically* on `calibration/`. We set it to the $(1 - \tau)$ quantile of the $P(\text{AI})$ scores of real calibration images with a target $\tau = 0.15$. This leaves a safe margin below the 0.20 limit for the hidden holdout. The threshold is never hand-set, and `validation/` is held out purely to verify the realised FPR.

Two model families. (1) *Classical baseline*: We engineered forensic features, which are a 32-bin radial FFT spectrum, colour/saturation statistics and high-pass residual statistics. These features were fed to a histogram gradient-boosted tree. It trains in seconds and, as Table 2 shows, reaches recall 0.699 at FPR 0.101 on validation. Its most important features (permutation importance) are the highest-frequency spectral bins and the high-vs-low-frequency contrast, confirming the Task 1.1 analysis (Fig. 4a). (2) *CNN from scratch*: We designed a compact VGG-style network with a 5×5 stride-2 stem (so the body runs at half resolution for CPU speed while the first layer still sees full 64px detail), BatchNorm and global average pooling ($\approx 430k$ parameters). Input is the squared RGB image at 64^2 ; inside the network we additionally concatenate a fixed high-pass residual (RGB minus a 3×3 blur) as extra channels, giving the detector an explicit forensic cue motivated by the Task 1.1 spectral analysis while keeping the external input plain RGB.

Efficient, reproducible training. The Appendix C reference takes 479s for 35 steps on our machine and our `train.py` runs in 1640s ($3.43 \times$ reference, within the $5 \times$ guideline) and well under the 1800s timeout. Because CPU speed varies, we adjusted the training loop to *measure* its own step time and sizes a one-cycle LR schedule to the number of epochs that fit the budget, so the learning rate anneals fully regardless of hardware; the best checkpoint (by validation recall subject to the FPR margin) is flushed whenever it improves, so an early timeout still yields a usable model. We use class-weighted cross-entropy (real weighted $\sim 3 \times$) so the minority real class is not ignored.

Results and ablations. Table 2 compares both families. The CNN reaches recall 0.802 at FPR 0.176 on validation, meeting the 0.8 recall target while keeping the FPR safely below the 0.20 limit, and shows a strong

Model	validation		validation_augmented	
	FPR _{real}	recall _{AI}	FPR _{real}	recall _{AI}
Classical (features + GBT)	0.101	0.699	0.262	0.474
CNN — Task 2 (clean)	0.176	0.802	0.225	0.664
CNN — Task 3 (augmented)	0.181	0.756	0.107	0.618

Table 2: Operating point (threshold auto-calibrated on **calibration/** to keep FPR safely below the 0.20 limit; Task 2 target 0.15, Task 3 a more conservative 0.14 for the augmented distribution). The constraint holds on *validation* for every row; the clean models violate it on **validation_augmented** (0.262/0.225), which is exactly what the augmented model repairs (0.107).

improvement over the classical baseline (recall 0.699). We made a few key choices that mattered. The first one was squaring the inputs by removing the aspect-ratio shortcut. Without it validation FPR collapses. The second one was BatchNorm (fast from-scratch convergence in the few affordable epochs). The third one was the stride-2 stem, which lets us keep 64px inputs within budget. The fourth choice was calibrating on a held-out split (the realised validation FPR tracks the target).

4 Data Augmentation and Feature Engineering

The task was to improve the detector’s robustness against real-world settings, to still meet the 0.2 FPR limit. We decided to continue from the Task 2 checkpoint and tested the model under altered conditions. This resulted in an FPR of 0.262, well above the allowed threshold.

Approach

To fix this, we took the transformations mentioned in the task (scaling, compression, blur) and added a few more common ones we felt were missing (brightness/contrast jitter, additive noise, horizontal flips), applying them randomly to each batch during training. We picked the best checkpoint based on a combined clean+augmented validation set, so we wouldn’t accidentally pick an early epoch that already performs badly on clean data. We also recalibrated the threshold on combined clean+augmented calibration data, using a slightly more conservative target ($\tau = 0.14$ instead of 0.15) to leave extra margin given the noisy FPR estimate from few real samples.

Results

Table 2 (last row) shows the augmented model lowers recall on **validation_augmented** from 0.664 (clean CNN) to 0.618 at FPR 0.107, so we are still clearing the 0.6 target and now meet the FPR limit of 0.2. The validation performance for the untransformed data is also affected, with a recall of 0.756 (previous 0.802) at FPR 0.181 but still meets all requirements.

5 Explainability

We looked at the trained CNN with three methods on validation images (Fig. 3): vanilla-gradient saliency, Grad-CAM on the last convolutional block, and occlusion sensitivity (we slide a neutral grey patch over the image and check how much $P(\text{AI})$ changes). We picked the most borderline cases for each category: true-positive (AI), true-negative (real), false-positive, and false-negative.

What we found

Saliency turned out pretty noisy and spread out over the whole image instead of pointing at one object. Grad-CAM often highlighted background or sky instead of the actual subject, which fits with a model that’s picking up on general texture statistics rather than what’s actually in the picture. We trust the occlusion result more than saliency or Grad-CAM here, since it directly measures the effect on the output rather than relying on gradients. The occlusion analysis was the most useful of the three: when we covered up a patch with grey, $P(\text{AI})$ almost always went up, meaning the model treats flat, low-detail regions as “AI-like”. Averaged over many images (Fig. 4b), this effect is strongest in the center of the image.

Interestingly, the occlusion effect is noticeably stronger on real images than on AI ones. This suggests the model’s decision for real images depends more heavily on local high-detail regions, whereas for AI images the ‘fake’ signal is spread more evenly across the whole image, consistent with AI images being globally smoother. This highlights two things the model is doing wrong: it confuses smoothness with “fake” (because AI images tend to be smoother), and it has a slight bias toward the center of the image. It also explains our errors: false positives are smooth, low-detail real photos, and false negatives are unusually detailed AI images. So overall, we shouldn’t fully trust these explanations. The model is relying on a real but shortcut-prone signal (local detail/smoothness), not something robust.

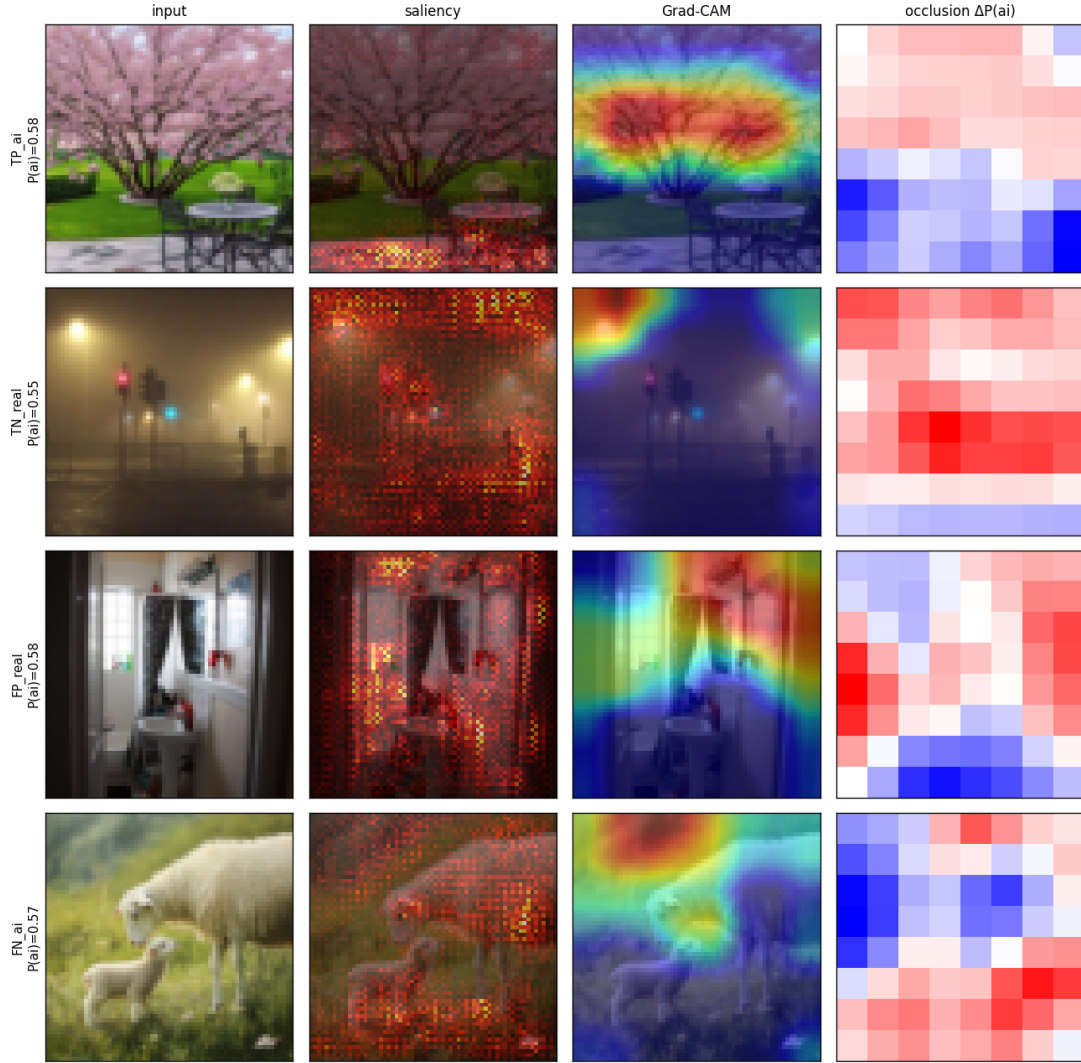
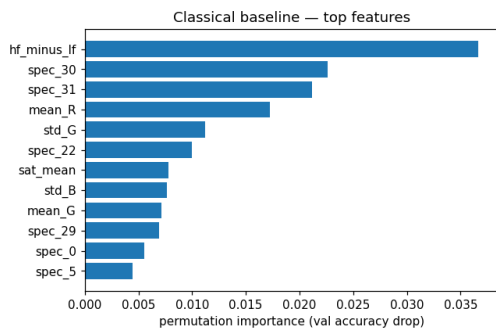


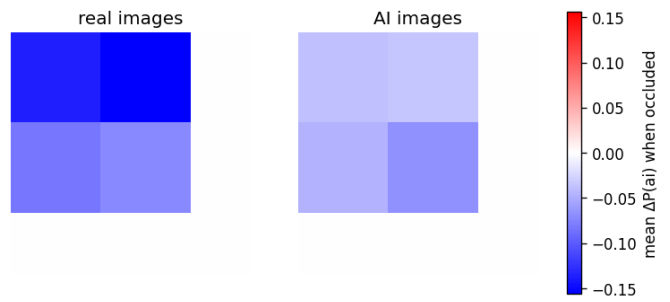
Figure 3: Per-case explanations (rows: borderline TP-AI, TN-real, FP, FN). Columns: input, saliency, Grad-CAM, occlusion $\Delta P(AI)$.

6 Conclusion

We found and fixed an aspect-ratio shortcut, hit the Task 2 target on augmented data, all within the same CPU budget. The main risks that are left: the calibration/validation splits only have a small number of real images, so the FPR estimate is a bit noisy (we tried to account for this with the calibration margin), and the model still has the center/smoothness biases we found in the explainability section.



(a) Classical feature importance.



(b) Mean occlusion importance (real vs. AI).

Figure 4: What the models rely on: high-frequency spectral bins (a) and a mild centre bias in spatial attention (b).